

05 - 11.
개체 지향
프로그래밍

목차

- 도입 배경
 - 데이터 관리의 어려움
- 개체
 - 상태와 행위를 가진 것
 - 자율성을 가진 것
 - 개체 정의하기
- 협력
 - 추상화: 메시지를 정의하는 도구
 - 상속과 다형성: 메시지를 처리하는 도구
 - 추상 클래스
 - sealed
- 동작 원리
 - 개체의 생성 순서
 - 사용자 정의 타입의 메모리 레이아웃
 - this
 - 가상 함수 테이블

도입 배경

데이터 관리의 어려움

- 데이터와 이를 조작하는 함수가 분리되어 있는 것은 **한계가 있다**.
 - **데이터의 무결성이 쉽게 훼손된다**: 데이터가 외부에 노출되어 있기 때문에 보호할 수가 없다.
 - **수정의 파급 효과를 감당하기 어렵다**: 데이터 구조가 바뀌면 코드의 여러 곳을 바꿔야 한다.
 - **인간의 사고 방식과는 거리가 멀다**: 올바르게 사용하기 위해 데이터와 함수를 짝꿍으로 기억하고 있어야 한다.
- 그래서 데이터와 이를 조작하는 함수를 묶은 단위인 **개체를 중심으로 프로그램을 구성**하기 위해 개체 지향 프로그래밍(OOP; Object-Oriented Programming)[1]이 등장했다.

[1]: 처음에는 Object를 객체로 번역했으나, 객체의 본래 의미는 Object하고는 맞지 않아 본 강의에서는 개체 지향 프로그래밍이라 부른다. 하지만, 아직 객체 지향 프로그래밍이라고 하는 곳이 많다.

개체

상태와 행위를 가진 것

- 프로그래밍 언어에서 제공하는 구조체, 클래스 등 **사용자 정의 타입**(User Defined Type)을 이용하여 여러 요구 사항을 개체로 모델링할 수 있다.
 - 사용자 정의 타입으로 개체의 템플릿을 정의하며, 이를 바탕으로 메모리에 표현(Representation)한 것을 **인스턴스**(Instance)라 한다.
- 개체는 **상태(데이터)**와 **행동/기능(함수)**을 함께 갖고 있으며, 상태는 **필드**(Field), 행동은 **메소드**(Method)라 부른다.

자율성을 가진 것

- 개체의 상태는 외부의 코드가 아니라 내부의 코드에 의해 수정되어야 한다.
- 내부의 상태[1]와 상세 구현을 숨기고, 외부에 노출시킬 행위를 함께 작성하는 것을 캡슐화(Encapsulation)라 한다.
 - 데이터의 무결성이 쉽게 훼손된다. → 캡슐화로 외부의 잘못된 조작으로 인해 객체의 데이터가 망가지는 것을 보호할 수 있다.
 - 수정의 파급 효과를 감당하기 어렵다. → 내부 구현이 바뀌어도 인터페이스가 바뀌지 않으면 다른 코드에 영향을 주지 않는다. 사용자 입장에서는 객체의 내부 구조를 알 필요 없이 노출된 행위만 사용하면 된다.
 - 인간의 사고 방식과는 거리가 멀다. → 상태와 행위를 함께 작성할 수 있다.

자율성을 가진 것

- 개체의 내부와 외부를 구분하기 위해서 접근 한정자(Access Modifier)를 사용한다.[1]
 - public 멤버는 타입 외부에서 접근 가능하며, 개체를 조작할 수 있는 행위를 제공하는데 사용한다.
 - private 멤버는 타입 내부에서만 접근 가능하며, 개체 내부에서 사용하는 헬퍼 메소드(Helper Method)나 외부에서 수정할 수 없는 필드를 만드는데 사용한다.
- 그래서 개체 지향의 세계에서는 서로의 메소드를 호출시키며 전체 시스템을 구현해 나간다.[2]

[1]: 참고로 C#에서 모든 타입과 타입 멤버의 기본 접근 한정자는 internal이다. internal은 같은 어셈블리 내에서만 접근할 수 있다. 이는 라이브러리 구현을 위한 것이다.

[2]: 이를 개체 지향에서는 '메시지를 보낸다'고 한다.

개체 정의하기

- 클래스 및 구조체에는 여러 가지 멤버(Member)를 정의할 수 있으며, 멤버마다 접근 한정자를 붙인다.
 - 필드(Field): 개체를 구성하는 데이터
 - 메소드(Method): 개체의 행위를 정의한 함수
 - 프로퍼티(Property): 필드처럼 사용할 수 있는 메소드
 - 이벤트(Event): 개체의 변화를 외부에 통지할 수 있는 대리자
- 개체의 초기화는 개체 초기자(Object Initializer) 혹은 생성자(Constructor)를 이용한다.
- 클래스는 같은 이름을 가진 파일에 작성하는 편이나, 필요한 경우 partial 클래스를 사용할 수 있다.

개체 정의하기

```
class Character
{
    // 필드부터 적는다.
    private int _maxHp;
    private int _currentHp;

    // 우리가 구현한 메소드를 통해 Character 타입을 다루게 된다.
    public void TakeDamage(int damage)
    {
        // 객체의 상태는 내부에서 조작해야 한다.
        _currentHp -= damage;
    }
}
```

개체 정의하기

- 멤버에는 각 인스턴스마다 독립적[1]으로 갖고 있는 **인스턴스 멤버**(Instance Member)와 모든 인스턴스가 공유하는 **정적 멤버**(Static Member)가 있다.[2]

```
class DrunkenObject
{
    // 정적 멤버는 static 키워드를 붙인다.
    // 정적 멤버는 모든 인스턴스에서 공유된다.
    private static int s_sharedValue = 0;

    public int Value => s_sharedValue;

    public void DoAwesome()
    {
        ++s_sharedValue;
    }
}
```

```
DrunkenObject drunken1 = new();
drunken1.DoAwesome();
```

```
DrunkenObject drunken2 = new();
drunken2.DoAwesome();
```

```
Console.WriteLine(drunken1.Value); // 2
```

[1]: 물론 실제로 독립적인 것은 상태(데이터) 뿐이다.

[2]: 정적 멤버만을 모아 놓는 **정적 클래스**(Static Class)도 있다.

개체 정의하기

- 기존 타입에 대한 재컴파일 없이 멤버를 확장할 수 있다.

```
// 보통 확장 메소드를 모아두는 정적 클래스를 만든다.
static class RegexExtensions
{
    // 확장 메소드는 반드시 정적 메소드여야 한다.
    // 확장 메소드의 첫 번째 매개변수는 확장시킬 타입의 매개변수여야 하며, 앞에 this 키워드를 붙인다.
    public static int Count(this Regex regex, string str)
    {
        MatchCollection matchCollection = regex.Matches(str);

        return matchCollection.Count;
    }
}

Regex regex = new Regex(@"(pPAp)");
string input = "ApPApPpAPpApPAp";

// 본래 Regex.Count()는 NET 7.0 이상에서 제공하나,
// NET 7.0이 아니더라도 Count()를 사용할 수 있게 됐다.
Console.WriteLine(regex.Count(input));
```

하루

추상화: 메시지를 정의하는 도구

- 개체는 각기 **고유한 책임**을 가진다. 즉, 협력에 참여하기 위해서 어떤 행동[1]을 수행한다.
- 그래서 단일의 개체로 프로그램을 만들 수 없다. 다시 말해 개체로 모듈화하여 전체 프로그램을 완성해야 한다.
- 개체는 **다른 개체에게 메시지를 보내며 서로 협력**한다.
- **개체끼리 서로 소통하기 위해 필요한 메시지를 정의하는 것을 추상화(Abstraction)**라 한다.
 - 메시지는 '어떻게(How)'는 숨기고 **'무엇을(What)'** 할 수 있는지 나타낸다.
 - 메시지는 수행 방법을 제한할 정도로 너무 구체적이어서는 안 되고, 협력의 의도를 명확히 표현하지 못할 정도로 추상적이어서도 안 된다.

[1]: 어떤 개체를 생성하거나, 어떤 연산을 수행하거나, 다른 개체의 메소드를 호출하는 것 등이 있다. 14

추상화: 메시지를 정의하는 도구

```
// Character 타입의 메시지 목록
```

```
Character
```

```
{
```

```
    // 메시지가 너무 구체적이다. 공격 수단이 다양해지면 각각에 대한 메소드를 정의해야 한다.
```

```
    void AttackWithAxe();
```

```
    // 메시지가 너무 추상적이다. '무엇을 해야 하는지' 모른다.
```

```
    // 코드의 가독성이 떨어지고, 설계 의도를 명확히 표현하지 못한다.
```

```
    void Do();
```

```
    // 적절한 수준의 추상화를 갖춘 메시지다.
```

```
    void Attack(IWeapon weapon);
```

```
}
```

메시지는 협력할 수 있는 수준으로 추상화가 이뤄져야 한다.

추상화: 메시지를 정의하는 도구

- C#에서는 이를 위해 인터페이스(Interface)를 제공한다.
 - 인터페이스는 추상 타입이므로 인스턴스를 생성할 수 없다.

```
// 인터페이스에는 I 접두사를 붙인다.  
public interface IEatable  
{  
    // 메소드의 헤더만 적는다.  
    // 기본 접근 한정자는 public이다.  
    void Eat();  
}
```

```
IEatable eatable = new IEatable(); // ! 불가능
```


상속과 다형성: 메시지를 처리하는 도구

- 인터페이스로 메시지를 정의했다면 이를 상속(Inheritance) 받아 자신만의 행동을 구현할 수 있다. 이를 오버라이딩 (Overriding)이라 한다.
- 코드를 물려받는 타입을 **자식**(Child)/**하위**(Sub)/**파생** (Derived) 타입, 물려주는 타입을 **부모**(Parent)/**상위**(Super)/**기본**(Base) 타입이라 한다.

```
// 자식 타입 : 부모 타입 형태로 작성한다.  
public class Human : IEatable  
{  
    public void Eat()  
    {  
        Console.WriteLine("사람이 먹는 중");  
    }  
}
```

```
public class Dog : IEatable  
{  
    public void Eat()  
    {  
        Console.WriteLine("개가 먹는 중");  
    }  
}
```

상속과 다형성: 메시지를 처리하는 도구

- 상속 관계에 있는 타입끼리는 캐스팅이 가능한데, 상위 클래스로 캐스팅하는 것을 **업캐스팅** (Upcasting), 하위 클래스로 캐스팅하는 것을 **다운캐스팅** (Downcasting)이라 한다.[1]
 - 캐스팅 연산자를 사용하지는 않고 as 연산자 혹은 is 연산자를 활용한다.
- **같은 인터페이스로 서로 다른 동작을 수행하는 것을 다형성** (Polymorphism)이라 한다.[2]

```
IEatable eatable = new Human(); // 업캐스팅
eatable.Eat(); // "사람이 먹는 중"
eatable = new Dog(); // 업캐스팅
eatable.Eat(); // "개가 먹는 중"
```

```
Dog? dog = eatable as Dog; // 다운캐스팅
if (dog is not null)
{
    // Dog 타입에 있는 메소드를 호출할 수 있다.
}
```

[1]: C++에서는 **사이드캐스팅** (Sidecasting)도 있다.

[2]: 다형성 덕분에 개체 지향에서는 불필요한 조건문이 사라지고 프로그램의 확장 용이성이 증대된다. 즉, 다형성이 OOP의 꽃이다. 18

추상 클래스

- abstract 키워드를 붙여 추상 클래스(Abstract Class)로 만들 수 있다.
 - 추상 클래스는 인터페이스와 마찬가지로 추상 타입이기에 인스턴스를 생성할 수 없다.
- 추상 클래스는 인터페이스와 다르게 **필드를 추가할 수 있다.**
- 인터페이스는 다중 상속이 가능하나, 클래스는 **단일 상속만 가능하다.**
- 가상 멤버(Virtual Member)[1] 혹은 추상 멤버로 하위 클래스에서 오버라이딩할 멤버를 지정할 수 있다.

```
// 추상 클래스는 abstract 키워드를 붙인다.
public abstract class SuperClass
{
    // 추상 멤버는 abstract 키워드를 붙인다.
    public abstract void Foo();
    // 가상 멤버는 virtual을 붙인다.
    public virtual void Boo()
    {
        // 가상 멤버는 기본 구현을 가질 수 있다.[2]
    }
}
```

[1]: 보통 가상 함수라고 부른다. 함수인 것들만 virtual을 붙일 수 있기 때문이다. 인터페이스에 정의한 멤버도 모두 가상 함수다.

[2]: 사실 기본 구현은 인터페이스에도 가능하다. 그러나, 구현 상속은 항상 신중해야 한다. 19

sealed

- 하위 클래스에서 오버라이딩 하는 것을 막고 싶다면 sealed 키워드를 붙인다.
- 클래스 혹은 멤버에 붙일 수 있다.

```
public sealed class SealedClass : SuperClass { } // 더이상 상속 불가능
public class DerivedClass : SuperClass
{
    // override를 붙여 오버라이딩한다.
    public override void Foo() { Console.WriteLine("DerivedClass::Foo"); }

    // 더이상 하위 클래스에서 오버라이딩을 금지할 수 있다.
    public sealed override void Boo()
    {
        // base 키워드로 상위 타입에 접근할 수 있다.
        base.Boo();
    }
}
```

동작 원리

개체의 생성 순서

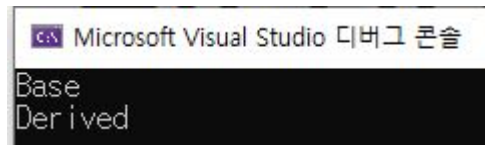
- 상속받을 때, 상위 타입부터 생성된다.

```
class Base
{
    private int a;

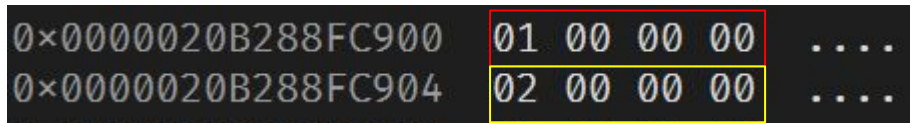
    public Base()
    {
        a = 1;
        Console.WriteLine("Base");
    }
}
```

```
class Derived : Base
{
    private int b;

    public Derived()
    {
        b = 2;
        Console.WriteLine("Derived");
    }
}
```



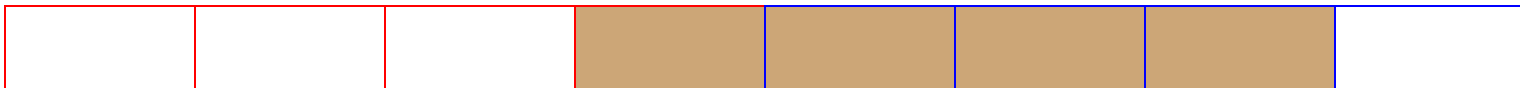
Base부터 출력된다.



인스턴스 생성 시 부모의 필드를 그대로 물려받는 것을 확인할 수 있다.

사용자 정의 타입의 메모리 레이아웃

- CPU가 메모리에서 데이터를 읽을 때는 데이터 버스의 대역폭만큼 읽는다.
- 단 한 번의 메모리 접근으로 데이터를 가져와 연산 속도를 높이기 위해 대부분 프로그래밍 언어는 메모리 정렬(Memory Alignment)을 수행한다.
 - C++ 같은 언어는 필드 사이에 의미 없는 빈 공간인 패딩(Padding) 바이트를 삽입해 각 필드를 정렬된 주소에 배치한다.
 - C#은 패딩을 최대한 줄이기 위해 필드를 재배치한다.[1]



데이터가 정렬된 주소에 배치되어 있지 않으면 2번에 걸쳐서 읽어야 한다.

[1]: 엄밀히는 class 타입만 그렇고, struct의 경우에는 필드를 재배치하진 않는다. 프로그래머가 직접 제어할 수도 있다. 23

사용자 정의 타입의 메모리 레이아웃

// Test의 실제 크기는 16바이트다.

```
class Test
```

```
{
```

```
    private int a = 1;
```

```
    private char b = 'b';
```

```
    private int c = 3;
```

```
    private byte d = 5;
```

```
    private char e = 'c';
```

```
}
```

0x0000010972062948	01 00 00 00	
0x000001097206294C	03 00 00 00	
0x0000010972062950	62 00 63 00	b.c.
0x0000010972062954	05 00 00 00	

프로그래머가 정의한 필드의 순서와 실제 메모리에 배치된 순서가 다르다.

this

- 인스턴스 메소드에는 항상 첫 번째 인수로 인스턴스의 참조가 전달되는데, 이를 this라 부른다.[1]

```
class Test
{
    private int a;
    private int b;

    public void Add(int value)
    {
        // 인스턴스 멤버에 접근할 때는 this가 생략된 것이다.
        this.b += value;

        // 이때문에 인스턴스 멤버에서 정적 멤버를 사용할 수 있지만
        // 역으로는 불가능하다.
        Print(a);
    }

    public static void Print(int value)
    {
        Console.WriteLine(value);
    }
}
```

[1]: 보통 this 포인터라고 부른다. 25

this

- 인스턴스 메소드에는 항상 첫 번째 인수로 인스턴스의 참조가 전달되는데, 이를 this라 부른다.

```
test.Add(2);  
00007FFB4B9D0807  mov     rcx,qword ptr [rbp+28h]  
00007FFB4B9D080B  mov     edx,2  
00007FFB4B9D0810  cmp     dword ptr [rcx],ecx  
00007FFB4B9D0812  call    Test.Add(Int32) (07FFB4BCD3BD0h)  
00007FFB4B9D0817
```

Add의 매개변수가 하나임에도 실제로 전달되는 인수의 개수는 2개다.

this

```
00007FFB4B9C1FAA  mov     qword ptr [rbp+40h],rcx
00007FFB4B9C1FAE  mov     dword ptr [rbp+48h],edx
00007FFB4B9C1FB1  cmp     dword ptr [7FFB4BC00948h],0
00007FFB4B9C1FB8  je      Test.Add(Int32)+01Fh (07FFB4B9C1FBFh)
00007FFB4B9C1FBA  call    00007FFBAB712590
00007FFB4B9C1FBF  nop
    // 인스턴스 멤버에 접근할 때는 this가 생략된 것이다.
    this.b += value;
00007FFB4B9C1FC0  mov     eax,dword ptr [rbp+48h]
00007FFB4B9C1FC3  mov     rcx,qword ptr [rbp+40h]
00007FFB4B9C1FC7  add     dword ptr [rcx+0Ch],eax

    // 이때문에 인스턴스 멤버에서 정적 멤버를 사용할 수 있지만
    // 역으로는 불가능하다.
    Print(a);
00007FFB4B9C1FCA  mov     rax,qword ptr [rbp+40h]
00007FFB4B9C1FCE  mov     ecx,dword ptr [rax+8]
00007FFB4B9C1FD1  call    Test.Print(Int32) (07FFB4BCC85E8h)
00007FFB4B9C1FD6  nop
```

this는 각 필드에 접근하기 위한 기준점이 된다.

가상 함수 테이블

- 함수를 호출하기 위해서는 **바인딩(Binding)**의 과정이 필요하다.
 - 바인딩은 식별자와 그에 해당하는 대상(속성, 값, 주소 등)을 연결하는 과정을 말한다.
- 일반 함수는 컴파일 시간에 호출할 함수의 주소가 결정되지만, 가상 함수는 **실행 시간에** 호출할 함수의 주소가 결정된다.[1]
- 그래서 실제로 호출될 함수의 주소를 모아둔 테이블을 **가상 함수 테이블(vtable)**이라 하며, **컴파일 시점에 클래스 당 하나씩** 생성된다.[2]
- 클래스에는 **가상 함수 테이블을 참조하는 숨겨진 첫 번째 필드(vptr)**가 있다. vptr은 객체가 생성될 때 초기화된다.

[1]: 전자를 **정적/초기 바인딩(Static Binding / Early Binding)**, 후자를 **동적/후기 바인딩(Dynamic Binding / Late Binding)**이라 한다.

[2]: C++에서는 가상 함수를 갖고 있는 클래스만 생성되는데, C#의 모든 타입은 System.Object를 상속 받기 때문에 모든 클래스가 가상 함수 테이블을 갖고 있다. 28

가상 함수 테이블

```
class Base
{
    public virtual void Foo() ⇒ Console.WriteLine("Base:: Foo");
    public void Boo() ⇒ Console.WriteLine("Boo");
}

class Derived : Base
{
    public override void Foo() ⇒ Console.WriteLine("Derived:: Foo");
}
```

```
obj.Boo();
00007FFB4BC80817 mov     rcx,qword ptr [rbp+28h]
00007FFB4BC8081B cmp     dword ptr [rcx],ecx
00007FFB4BC8081D call    Base.Boo() (07FFB4BF83C30h)
00007FFB4BC80822 nop

obj.Foo();
00007FFB4BC80823 mov     rcx,qword ptr [rbp+28h]
00007FFB4BC80827 mov     rax,qword ptr [rbp+28h]
00007FFB4BC8082B mov     rax,qword ptr [rax]
00007FFB4BC8082E mov     rax,qword ptr [rax+40h]
00007FFB4BC80832 call    qword ptr [rax+20h]
```

가상 함수인 Foo()는 실행 시점에 호출되는 것을 확인할 수 있다.

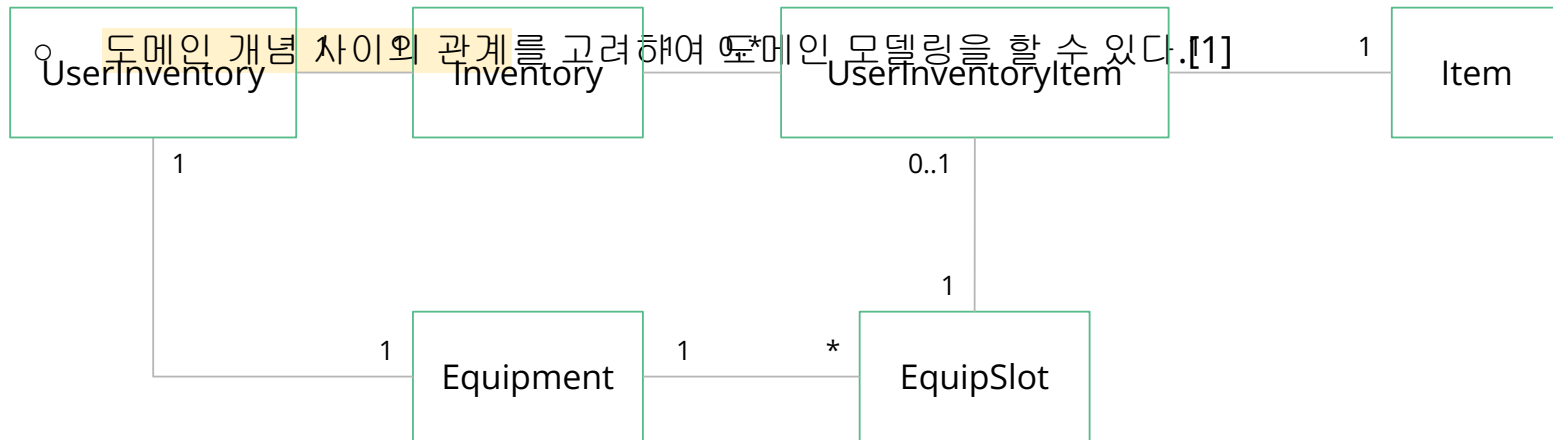
설계 가이드

현재에 집중하고, 수정이 가장 쉬운 설계를 택한다.

- 처음부터 모든 상황을 상정하고 완벽한 설계를 하려고 하지 말라.
 - 미래를 알 수 있는 프로그래머는 없다.
- 가독성을 챙기면 유지보수는 자연스레 따라온다.
- 코드 자체가 기획서와 같도록 표현적 차이(Representational Gap)를 줄인다.

도메인 모델링을 올바르게 하라.

- 코드를 작성하기 전, 도메인 모델링(Domain Modeling)부터 올바르게 해야 한다.
 - 도메인 (Domain)이란 소프트웨어로 구현할 요구 사항의 범위다.
 - 도메인 내 중요한 객체, 역할, 행위, 규칙 등을 추상화한 개념을 **도메인 개념 (Domain Concept)**이라 한다.



RPG에서 사용하는 유저 인벤토리의 도메인 모델링 예시

책임을 잘 할당하자.

- 일반적인 책임 할당을 위한 소프트웨어 패턴(GRASP; General Responsibility Assignment Software Pattern)을 참고한다.
 - **정보 전문가 (Information Expert):** 책임을 수행하는 데 필요한 정보를 가장 많이 알고 있는 개체에게 책임을 부여한다.
 - **창조자 (Creator):** 새로운 인스턴스를 생성할 책임을, 해당 인스턴스를 포함하거나, 참조하거나, 긴밀하게 사용하거나, 초기화에 필요한 정보를 알고 있는 개체에게 맡긴다.
 - **다형성 (Polymorphism):** 유사하지만 타입에 따라 다르게 행동해야 할 때, 조건문을 사용하지 않고 동일한 요청에 대해 타입별로 다른 행동을 수행하도록 책임을 할당한다.
 - **변경 보호 (Protected Variations):** 변화가 예상되거나 불안정한 지점을 식별하고, 추상화를 통해 다른 요소에 해로운 영향을 미치지 않도록 책임을 부여한다.

책임을 잘 할당하자.

- 일반적인 책임 할당을 위한 소프트웨어 패턴(GRASP; General Responsibility Assignment Software Pattern)을 참고한다.
 - **간접화 (Indirection):** 직접적으로 의존하지 않고 개체 간 중재하는 중간 개체를 만든다.
 - **컨트롤러 (Controller):** UI 계층을 통해 전달되는 사용자의 입력을 조정할 최초의 객체를 만든다.
 - **순수한 가공물 (Pure Fabrication):** 높은 응집도[1]와 낮은 결합도[2]를 유지하기 위해, 도메인 개념을 표현하지 않는 인위적으로 만든 클래스(기술적인 책임을 전담)에 책임을 할당한다.

[1]: **응집도(Cohesion)**는 '한 요소의 책임들이 얼마나 강력하게 관련되고 집중되어 있는가'를 말한다. 응집도는 높을수록 좋다. 응집도를 높이려면 함께 변경되거나 사용되는 메소드 및 데이터를 잘 확인하여 클래스로 분리한다.

[2]: **결합도(Coupling)**는 '모듈이 외부의 다른 모듈에 의존하는 정도가 낮음'을 의미한다. 결합도는 낮을수록 변경에 따른 파급 효과가 줄어들고 재사용성이 높아진다. 결합도를 낮추기 위해서는 추상화에 의존하도록 코드를 작성한다. }4

책임을 잘 할당하자.

- 단일 책임 원칙을 지키고 있는지 확인하자.
 - 객체의 책임을 서술하는데 **접속사**가 필요하다면 너무 큰 책임을 갖고 있는 것이다.
 - **일부 인스턴스 필드가 일부 메서드에 의해서만 사용된다면** 단일의 클래스로 묶을 수 있다.
 - **서로 배타적으로 초기화되는 인스턴스 변수가 있다면** 분류할 수 있다.
 - **수많은 private 메소드가 있는 것 또한 여러 책임을 갖고 있는 것일 수도 있다.**
 - 외부 의존성이 있다면 **의존성 주입**으로 처리할 순 없는지 고민한다.

값 개체를 활용하라.

- 시스템 내부에서 값을 표현하는 개체 (Value Object)는 불변 개체로 다룬다.(e.g. HP, MP, Exp, Gold 등)
- 동등성 (Equality)을 갖고 있다면 값 객체다.
- 초기화 후 필드에 상수성을 부여하기 위해 readonly를 사용한다.

```
public struct Health
{
    private readonly int _current;
    private readonly int _max;

    public int Current => _current;
    public int Max => _max;
    public float Normalized => (float)Current / Max;
    public bool IsDead => Current <= 0;

    public Health(int maxAmount) : this(maxAmount, maxAmount) { }
    public Health(int current, int max)
    {
        _current = current;
        _max = max;
    }

    public Health TakeDamage(int damage)
    {
        if (damage <= 0) return this;
        return new Health(Current - damage, Max);
    }

    public Health Heal(int amount)
    {
        if (amount <= 0) return this;
        return new Health(Current + amount, Max);
    }
}
```

완전한 개체를 생성하라.

- 개체는 메모리에 로드된 순간부터 즉시 사용 가능해야 한다. 즉, **항상 유효한 상태**를 유지해야 한다.
- 개체의 초기화가 생성자에서 끝나지 않고 여러 단계로 나뉘어진다면 버그가 발생하기 쉽다.[1]
 - 오류 상태로 개체가 존재할 수 있다.
 - 개체를 사용하기 전, 반드시 초기화를 하는 함수를 호출해야 한다는 순서를 개발자가 기억해야 한다.
- 가능한 생성자에서 초기화를 끝내자.

[1]: 그러나 아직 게임 업계에서는 2단계 초기화를 진행하는 경우가 많다. }7

완전한 개체를 생성하라.

```
public class UserProfile
{
    public string Name { get; private set; } // 필수 사항[1]
    public int Age { get; private set; }

    // 사용 전, Init()을 호출해야 한다.
    public bool Init(string name, int age)
    {
        if (string.IsNullOrEmpty(name))
        {
            return false;
        }

        Name = name;
        Age = age;

        return true;
    }

    public void PrintInfo()
    {
        Console.WriteLine($"{Name} ({Age})");
    }
}
```

// 사용 예시

```
UserProfile userProfile = new UserProfile();
userProfile.PrintInfo(); // [!] 사용 전, Init() 호출을 하지 않음.
```

완전한 개체를 생성하라.

- 개체는 메모리에 로드된 순간부터 즉시 사용 가능해야 한다. 즉, **항상 유효한 상태**를 유지해야 한다.
- 개체의 초기화가 생성자에서 끝나지 않고 여러 단계로 나뉘어진다면 버그가 발생하기 쉽다.[1]
 - 오류 상태로 개체가 존재할 수 있다.
 - 개체를 사용하기 전, 반드시 초기화를 하는 함수를 호출해야 한다는 순서를 개발자가 기억해야 한다.
- 가능한 생성자에서 초기화를 끝내자.
- 초기화 단계가 복잡하거나 생성 실패를 보고해야 한다면 빌더 패턴(Builder Pattern) 혹은 팩토리 패턴(Factory Pattern)을 사용한다.

[1]: 그러나 아직 게임 업계에서는 2단계 초기화를 진행하는 경우가 많다. 39

개체의 상태를 묻기보다 작업을 시켜라.

- Getter / Setter를 만드는 것은 개체의 자율성을 저해하며 코드의 응집도를 떨어뜨리고, 결합도를 높인다.
- 개체의 상태를 가져와서 로직을 구성하지 말고 **개체의 메소드를 호출함**으로 로직을 구성하라.[1]

```
public class Player
{
    public Health Hp { get; set; }
}

public class DamageHandler
{
    public void DealDamage(Player player, int damage)
    {
        // 1. 객체의 데이터를 물어봄 (Ask)
        int currentHp = player.Hp.Current;

        // 2. 외부에서 로직을 처리함 (판단)
        int newHp = currentHp - damage;

        // 3. 다시 객체에 값을 주입함
        player.Hp = new Health(newHp, player.Hp.Max);

        if (player.Hp.Current <= 0)
        {
            Debug.Log("사망 처리");
        }
    }
}
```

[1]: TDA(Tell, Don't Ask) 원칙이라고 한다.

개체의 상태를 묻기보다 작업을 시켜라.

- Getter / Setter를 만드는 것은 개체의 자율성을 저해하며 코드의 응집도를 떨어뜨리고, 결합도를 높인다.
- 개체의 상태를 가져와서 로직을 구성하지 말고 개체의 메소드를 호출함으로 로직을 구성하라.[1]

```
public class Player
{
    public Health Hp { get; private set; }
    public bool IsDead => Hp.IsDead;
    public event Action OnDead;

    public void TakeDamage(int damage)
    {
        // 내부 데이터의 무결성은 Health 값 객체와 Player가 스스로 책임짐
        Hp = Hp.TakeDamage(damage);

        if (Hp.IsDead)
        {
            OnDead?.Invoke();
        }
    }
}

public class Enemy
{
    public void Attack(Player target)
    {
        // 복잡하게 계산하지 않고 그냥 명령만 함 (Tell)
        target.TakeDamage(10);
    }
}
```

[1]: TDA(Tell, Don't Ask) 원칙이라고 한다.

의존성을 관리하라.

- 어떤 클래스가 기능을 수행하기 위해 다른 클래스를 참조하거나 사용하는 것을 의존성(Dependency)이라 한다.
- 개체 지향에서는 개체 간 협력해야 하기에 의존성은 필연적으로 생긴다.
- 그러나, 의존한다는 것은 다른 말로 변경의 영향을 받는다는 것이다.
- 따라서 의존성을 관리하지 않으면 유지보수가 힘들어진다.

의존성을 관리하라.

- 개체 간 의존성은 인터페이스에 드러나야 한다.[1]

```
// 나쁜 예: 캐릭터가 공격할 때 소리를 내야한다면?  
public class Warrior  
{  
    // Warrior가 WavSoundPlayer에 의존한다는 사실을 코드 내부를 봐야 알 수 있다.  
    // 즉, 둘 사이에는 강한 결합이 존재한다.  
    // 추후 다른 사운드 플레이어에 필요하다면 코드를 수정하고 재컴파일 해야 한다.  
    private readonly WavSoundPlayer _soundPlayer = new WavSoundPlayer();  
  
    public void Attack()  
    {  
        Console.WriteLine("전사가 칼을 휘두릅니다!");  
        _soundPlayer.Play("sword_swing.wav");  
    }  
}
```

의존성을 관리하라.

- 개체 간 의존성은 인터페이스에 드러나야 한다.[1]

// 좋은 예: 의존성 주입으로 인터페이스에 의존성을 드러낸다.

// 전사는 무엇이든 소리만 낼 수 있으면 된다.

```
public interface ISoundProvider
{
    void Play(string fileName);
}
```

// 각각 생성

```
public class WavPlayer : ISoundProvider
{
    public void Play(string fileName) => Console.WriteLine($"[WAV 재생] {fileName}");
}
```

```
public class Mp3Player : ISoundProvider
{
    public void Play(string fileName) => Console.WriteLine($"[MP3 재생] {fileName}");
}
```

의존성을 관리하라.

- 개체 간 의존성은 인터페이스에 드러나야 한다.[1]

```
public class Warrior
{
    private readonly ISoundProvider _soundProvider;

    // 이제는 Warrior와 ISoundProvider 간 의존성이 인터페이스에 드러난다.
    // Warrior는 안정적인 추상화에 의존하므로 유연성이 확보된다.
    // 외부에서 언제든지 다른 사운드 플레이어로 교체할 수 있다.
    public Warrior(ISoundProvider soundProvider)
    {
        _soundProvider = soundProvider ?? throw new ArgumentNullException(nameof(soundProvider));
    }

    public void Attack()
    {
        Console.WriteLine("전사가 칼을 휘두릅니다!");
        _soundProvider.Play("attack_swing");
    }
}
```

의존성을 관리하라.

- 개체가 직접 의존하는 개체하고만 소통해야 한다.[1]

// 나쁜 예: 플레이어가 아이템을 사용하면 인벤토리의 특정 슬롯에 있는 아이템의 내구도를 깎아야 할 때

```
public class Player
{
    public Inventory Inventory { get; set; } = new Inventory();
}
```

```
public class Inventory
{
    public List<Item> Items { get; set; } = new List<Item>();
}
```

```
public class Item
{
    public int Durability { get; set; }
}
```

[1]: 디미터 법칙(Law of Demeter) 혹은 최소 지식 원칙(Principle of Least Knowledge)이라 한다. 46

의존성을 관리하라.

- 개체가 직접 의존하는 개체하고만 소통해야 한다.[1]

```
// --- 사용하는 곳 (문제의 코드) ---  
public class GameSystem  
{  
    public void RepairItem(Player player, int slotIndex)  
    {  
        // 디미터 법칙 위반: player -> inventory -> items -> durability  
        // 플레이어의 가방 속의 아이템의 내구도를 직접 건드린다.  
        // 이를 기차 충돌(Train Wreck)이라 부른다.  
        player.Inventory.Items[slotIndex].Durability += 10;  
    }  
}
```

의존성을 관리하라.

- 개체가 직접 의존하는 개체하고만 소통해야 한다.[1]

// 좋은 예: TDA 원칙을 지켜라.

```
public class GameSystem
```

```
{
```

```
    public void RepairItem(Player player, int slotIndex)
```

```
    {
```

```
        // 이제 GameSystem은 플레이어만 알면 됨. 내부 구조는 몰라도 됨.
```

```
        player.RepairItem(slotIndex, 10);
```

```
    }
```

```
}
```


의존성을 관리하라.

- 개체가 직접 의존하는 개체하고만 소통해야 한다.[1]

```
public class Player
{
    private Inventory _inventory = new Inventory();

    // 플레이어에게 수리를 요청함
    public void RepairItem(int slotIndex, int amount)
    {
        _inventory.RepairItem(slotIndex, amount);
    }
}
```

의존성을 관리하라.

- 개체가 직접 의존하는 개체하고만 소통해야 한다.[1]

```
public class Inventory
{
    private List<Item> _items = new List<Item>();

    // 인벤토리에게 수리를 요청함
    public void RepairItem(int slotIndex, int amount)
    {
        if (slotIndex < _items.Count)
        {
            _items[slotIndex].IncreaseDurability(amount);
        }
    }
}
```

의존성을 관리하라.

- 개체가 직접 의존하는 개체하고만 소통해야 한다.[1]

```
public class Item
{
    public int Durability { get; private set; }

    // 아이템 스스로 내구도를 올리게 함
    public void IncreaseDurability(int amount)
    {
        Durability += amount;
    }
}
```

[1]: 디미터 법칙(Law of Demeter) 혹은 최소 지식 원칙(Principle of Least Knowledge)이라 한다. 5]

올바른 계층 관계를 구성하라.

- 올바른 상속은 코드의 중복을 방지하기 위해 사용하는 것이 아니라 올바른 계층 관계를 수립하는 것이다.[1]
 - 하위 타입을 업캐스팅하여 사용했을 때 상위 타입의 기대를 저버려서는 안 된다.
- 코드 중복을 피하는 목적으로 상속을 사용하면 부모와 자식이 강하게 결합하여 유지보수가 힘들어진다.
 - 구현 상속(Implementation Inheritance)보다는 인터페이스 상속(Interface Inheritance)을 사용해야 한다.[2]
 - 단 하나의 구현만을 포함하라.[3]
 - 코드 중복을 피하고 싶다면 상속 대신 합성(Composition)을 사용한다.[4]
- 상속 계층은 얇고 넓은 것을 지향한다.
 - 3단계 이상 깊어지면 코드를 이해하기 어려워진다.

[1]: 올바른 계층 구조를 갖고 있는 부모와 자식은 **is-a** 관계를 갖는다.

[2]: 다르게 얘기하면 서브클래싱(Subclassing)이 아니라 서브타이핑(Subtyping)을 목표로 한다.

[3]: 이러한 계층을 정규화된 계층(Normalized Hierarchy)이라 한다.

[4]: 합성의 관계를 **has-a** 관계라 한다. 52

올바른 계층 관계를 구성하라.

```
public class Rectangle
{
    public virtual int Width { get; set; }
    public virtual int Height { get; set; }
    public int GetArea() => Width * Height;
}
```

```
public class Square : Rectangle
{
    // 정사각형은 가로/세로가 항상 같아야 하므로 한쪽만 바뀌도 둘 다 바꿈
    public override int Width { set { base.Width = base.Height = value; } }
    public override int Height { set { base.Width = base.Height = value; } }
}
```

```
Rectangle rect = new Square();
rect.Width = 5; // [!] Height도 바뀐다.
```

정사각형은 직사각형이지만, 굳이곧대로 코드에 적용하면 문제가 생긴다.

올바른 계층 관계를 구성하라.

```
// 1. 기능을 인터페이스로 쪼갭니다 (Role-based)
public interface IMoveBehavior { void Move(); }
public interface IAttackBehavior { void Attack(); }

// 2. 구체적인 기능들을 만듭니다
public class Walk : IMoveBehavior { public void Move() => Console.WriteLine("걷기"); }
public class Fly : IMoveBehavior { public void Move() => Console.WriteLine("비행"); }
public class SwordSwing : IAttackBehavior { public void Attack() => Console.WriteLine("칼 휘두르기"); }

// 3. 유닛은 상속이 아니라 '조합'으로 만듭니다 (Composition)
public class GameUnit
{
    private readonly IMoveBehavior _moveBehavior;
    private readonly IAttackBehavior _attackBehavior;

    // 생성 시점에 어떤 기능을 가질지 주입받음 (강건한 객체 생성)
    public GameUnit(IMoveBehavior move, IAttackBehavior attack)
    {
        _moveBehavior = move;
        _attackBehavior = attack;
    }

    public void PerformMove() => _moveBehavior.Move();
    public void PerformAttack() => _attackBehavior.Attack();
}
```

코드 중복을 피하고 싶다면 전략 패턴(Strategy Pattern)과 합성을 이용해보라.

SOLID 원칙

- **단일 책임 원칙 (SRP; Single Responsibility Principle):** 변경의 이유가 하나여야 한다.
- **개방-폐쇄 법칙 (OCP; Open-Closed Principle):** 확장에는 열려 있고, 변경에는 닫혀 있어야 한다.
 - 기존 코드를 수정하지 않고 새로운 기능을 추가할 수 있어야 한다.
- **리스코프 치환 원칙 (LSP; Liskov Substitution Principle):** 하위 타입이 상위 타입을 치환해도 프로그램 동작이 변하지 않아야 한다.

SOLID 원칙

- **인터페이스 분리 원칙 (ISP; Interface Segregation):** 사용하지 않는 메서드에 의존하도록 강제해서는 안 된다.
 - 거대한 단일의 인터페이스가 아니라 꼭 필요한 메시지만 정의되어 있는 작은 인터페이스 여러 개가 낫다.
- **의존성 역전 원칙 (DIP; Dependency Inversion):** 고수준 모듈이 저수준 모듈의 구현에 의존해서는 안 된다.
 - 본질적인 목적과 관련이 깊으면 고수준, 거리가 멀면 저수준이다.
 - 입출력과 가까울수록 저수준이다.
 - 추상화로 모듈 간에 격리가 이루어져야 한다.

부록

더 나아가기

- C#에서 구조체와 클래스의 차이점은 무엇인가요?
- 개체 지향 프로그래밍을 정의하고, 장점을 서술해 보세요.
- 개체 지향 프로그래밍의 4대 특징을 서술해 보세요.
- 개체를 작성할 때 유념해야 할 것은 무엇인가요?
- 메모리 정렬과 패딩에 대해서 설명해 보세요.
- this는 무엇인가요?
- 가상 함수의 동작 원리에 대해서 설명해 보세요.

참고자료

- [오브젝트 - 기초편 강의 | 조영호 - 인프런](#)
- [오브젝트 - 설계 원칙편 강의 | 조영호 - 인프런](#)
- [궁극적인 소프트웨어 설계 원칙](#)
- [OOP에서 진짜 중요한 건 '캡슐화'도 '상속'도 아닙니다](#)
- [\[제목\] C# Virtual Table의 구조와 Polymorphism에 관한 이해](#)
- [클린 아키텍처 - YES24](#)
- [Responsibility-driven design - Wikipedia](#)
- [Class-responsibility-collaboration card - Wikipedia](#)
- [빌더 패턴](#)
- [팩토리 메서드 패턴](#)